ma4144inclass1-basics

August 19, 2025

#

ML Basics

##

Inclass Project 1 - MA4144

##

210031H A.A.W.L.R.Amarasinghe

This project contains 12 tasks/questions to be completed.

After finishing project run the entire notebook once and **save the notebook as a pdf** (File menu -> Save and Export Notebook As -> PDF). You are **required to upload this PDF on moodle**.

Use this cell to use any include any imports

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

0.1 Section 1

In this section we will analyse a dataset to look into its statistical propeties. For this we will use the DiabetesTrain.csv dataset. Each row corresponds to a single patient. The first 8 columns correspond to the features of the patients that may help predict risk of diabetes. The outcome column is a binary column represting the risk of diabetes, outcome 1 : high risk of diabetes and outcome 0 little to no risk of diabetes.

Q1. Read the dataset into a pandas dataframe called diabetesData.

[2]:
```
diabetesData = pd.read_csv('DiabetesTrain.csv')

# Display the first few rows of the dataset
diabetesData.head()
```

[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	1	95	74	21	73	25.9	
1	1	95	82	25	180	35.0	
2	1	90	68	8	0	24.5	

```
3            7      195             70             33       145  25.1
4            0      180             66             39         0  42.0

   DiabetesPedigreeFunction  Age  Outcome
0                     0.673   36        0
1                     0.233   43        1
2                     1.138   36        0
3                     0.163   55        1
4                     1.893   25        1
```

Q2. Let us define the following events.

A = Patient has BMI less than 25

B = Patient has Glucose level greater than 100

C = Patient has had more than 2 pregnancies

D = Patient has high risk of diabetes

Based on the above definitions determine the following probabilities. Pandas dataframe inbuilt functions such as count, group by, would be useful for this task.

[3]:
```python
#length of the dataset
n = len(diabetesData)
print(f"Number of records in the dataset: {n}")

#P(A)
P_A = (diabetesData['BMI'] < 25).sum() / n

#P(B)
P_B = (diabetesData['Glucose'] > 100).sum() / n

#P(C)
P_C = (diabetesData['Pregnancies'] > 2).sum() / n

#P(D)
P_D = (diabetesData['Outcome'] == 1).sum() / n

#print probabilities
print(f"P(A): {P_A}")
print(f"P(B): {P_B}")
print(f"P(C): {P_C}")
print(f"P(D): {P_D}")

#P(A,D)
P_A_D = ((diabetesData['BMI'] < 25) & (diabetesData['Outcome'] == 1)).sum() / n

#P(B,D)
```

```
P_B_D = ((diabetesData['Glucose'] > 100) & (diabetesData['Outcome'] == 1)).
 ↪sum() / n

#P(C,D)
P_C_D = ((diabetesData['Pregnancies'] > 2) & (diabetesData['Outcome'] == 1)).
 ↪sum() / n

# Compute conditional probabilities P(D|A), P(D|B), P(D|C)
P_D_given_A = P_A_D / P_A if P_A > 0 else 0
P_D_given_B = P_B_D / P_B if P_B > 0 else 0
P_D_given_C = P_C_D / P_C if P_C > 0 else 0

# print conditional probabilities
print(f"P(D|A): {P_D_given_A}")
print(f"P(D|B): {P_D_given_B}")
print(f"P(D|C): {P_D_given_C}")

# Determine which event contributes most to high risk of diabetes
max_contrib = max(P_D_given_A, P_D_given_B, P_D_given_C)
if max_contrib == P_D_given_A:
    Q2 = 'A'
elif max_contrib == P_D_given_B:
    Q2 = 'B'
else:
    Q2 = 'C'

# Print the result
print(f"The event that contributes most to the high risk of diabetes is: {Q2}")
```

```
Number of records in the dataset: 399
P(A): 0.14786967418546365
P(B): 0.7368421052631579
P(C): 0.5388471177944862
P(D): 0.37844611528822053
P(D|A): 0.05084745762711865
P(D|B): 0.48299319727891155
P(D|C): 0.46976744186046504
The event that contributes most to the high risk of diabetes is: B
```

Q3. Now we will compute the covariance and correlation matrices from scratch. For this do not use any inbuilt functions. Follow the steps outlined below. Each step is graded.

Step1: Convert the diabetesData dataframe into a 2-dimensional numpy array with the same number of rows and columns as in the dataframe. Name it diabetesX.

[4]:
```
diabetesX = diabetesData.values
```

Step2: In diabetesX; center every column, by subtracting each column by the column mean and reassign it to diabetesX.

[5]:
```
diabetesX = diabetesX - np.mean(diabetesX, axis=0)
```

Step3: Compute the covariance matrix. Use, matrix operations in numpy such as matrix multiplication, matrix transpose and don't forget to average. Assign it to the variable cov.

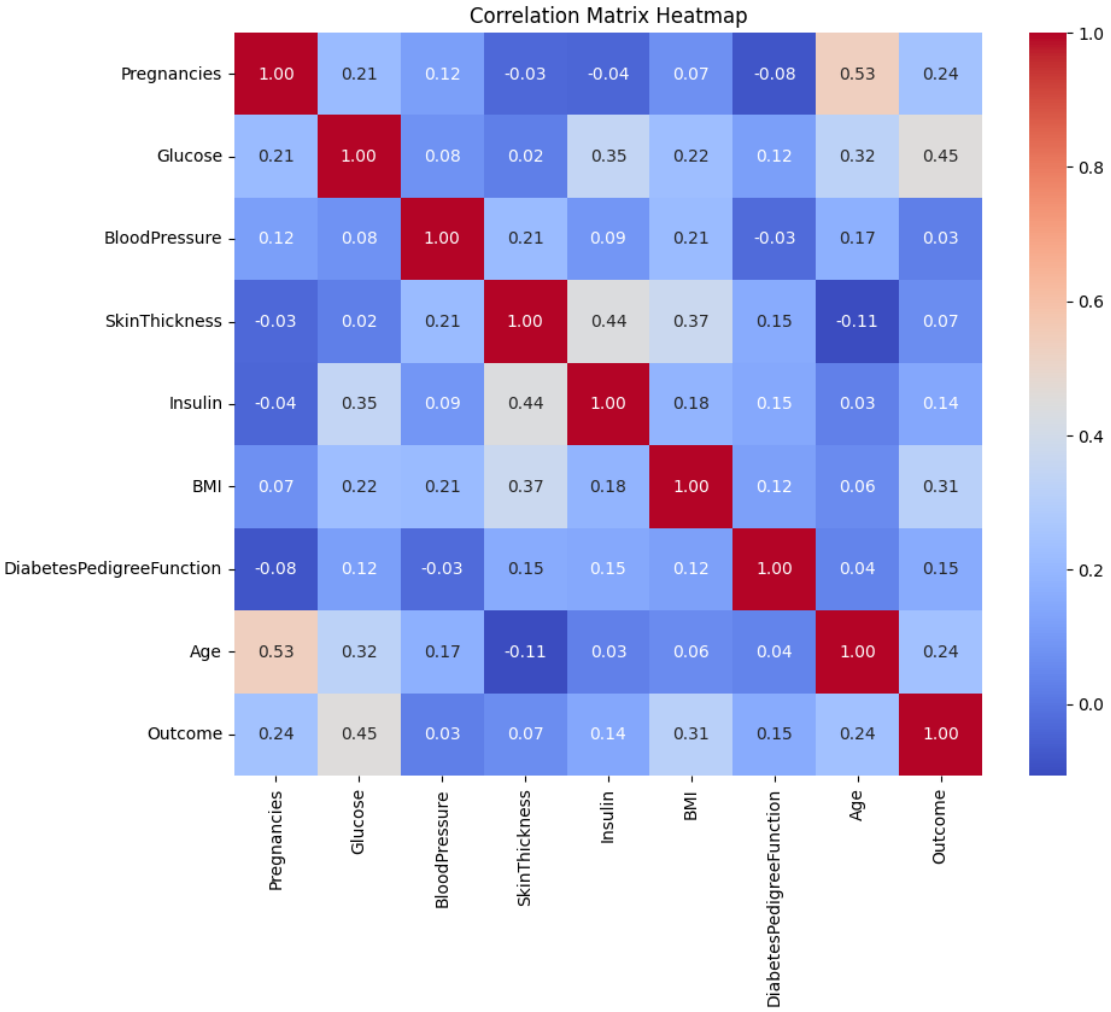
[6]:
```
n = diabetesX.shape[0]
cov = (diabetesX.T @ diabetesX) / n
```

Step4: Compute the matrix varmat, whose (i, j) entry is $\sqrt{var(i)var(j)}$, where $var(k)$ is the variance of the kth column of the diabetesX matrix. The varinces can be extracted from the covariance matrix itself appropriately and varmat can be computed by appropriate matrix multiplication of a column matrix and a row matrix.

[7]:
```
variances = np.diag(cov)
varmat = np.sqrt(np.outer(variances, variances))
```

Step5: Use the cov matrix and varmat matrix appropriately to compute the correlational matrix corr. And then use seaborn (sns imported above) to plot an annotated heatmap of the correlation matrix.

[8]:
```
# Compute the correlation matrix from cov and varmat
corr = cov / varmat

# Plot the annotated heatmap of the correlation matrix
plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',␣
 ↪xticklabels=diabetesData.columns, yticklabels=diabetesData.columns)
plt.title('Correlation Matrix Heatmap')
plt.show()
```



From the heatmap read the following correlations. Also, answer the question below.

[9]:
```
#Corr(BMI, Outcome)
Corr1 = 0.31

#Corr(Glucose, Outcome)
Corr2 = 0.45

#Corr(Pregnancies, Outcome)
Corr3 = 0.24

#Out of the 8 features, which two features are the most correlated. Fill in the␣
 ↪list variable bestcorr
bestcorr = ['Pregnancies', 'Age']

# Print the results
```

```
print(f"Correlation(BMI, Outcome): {Corr1}")
print(f"Correlation(Glucose, Outcome): {Corr2}")
print(f"Correlation(Pregnancies, Outcome): {Corr3}")
print(f"Most correlated features: {bestcorr}")
```

```
Correlation(BMI, Outcome): 0.31
Correlation(Glucose, Outcome): 0.45
Correlation(Pregnancies, Outcome): 0.24
Most correlated features: ['Pregnancies', 'Age']
```

0.2 Section 2

In this section we will train a logistic regression model on the diabetes dataset to predict the risk of diabetes given patient data. We will then use it to perform predictions on previously unseen (test) data.

Q4. Preprocess Data.

Implement a function normalizeData that normalizes each column of a data array. Recall that a column x is normalized by $z = \frac{x - mean(x)}{std(x)}$. The function should return the normalized data matrix and the mean and standard deviations of each column (we'll need these to normalize the test data later).

[10]:
```
def normalizeData(X, mean = np.array([]), std = np.array([])):
    # Implement such that if the mean and std are empty arrays then mean and␣
 ↪std is calculated here, else use the mean and std passed in as parameters
    if mean.size == 0:
        mean = np.mean(X, axis=0)
    if std.size == 0:
        std = np.std(X, axis=0)
    normalized_X = (X - mean) / std
    return normalized_X, mean, std
```

Q5. Sigmoid function and it's derivative.

1. Implement the sigmoid function. Given a numpy array, compute the sigmoid values of the array. It should return an array of sigmoid values. If you see any overflow errors/warnings popping up, that is because the numbers coming out of the exponential function e^{-t} in the sigmoid function are too large to handle. In that case, please clip the input between some large enough negative and positive value (look into numpy.clip).
2. Implement the derivative of the sigmoid function. Recall that if, $p = \sigma(t)$, then $p' = p(1-p)$. The function should take in an array of p values and then return the array of p'.

[11]:
```
def sigmoid(t):
    # Clip input to avoid overflow in exp
    t_clipped = np.clip(t, -500, 500)
    sig = 1 / (1 + np.exp(-t_clipped))
    return sig
```

[12]:
```
def derivSigmoid(p):
    # Derivative of sigmoid: p * (1 - p)
    deriv_p = p * (1 - p)
    return deriv_p
```

Q6. Compute sigmoid probabilities.

Recall that $p = P(y = 1|x) = \sigma(x^T\omega + b)$, where $\omega = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \vdots \\ \omega_d \end{bmatrix}$ are the weights and b is the bias term.

Implement a function sigProg to calculate those probabilities, given a matrix X of data, weight vector ω and bias b. It is possible to compute the array of probabilities for the entire dataset at once without a single for-loop, by just using matrix mutltiplications, which is the rightway to achieve maximum efficiency. You are supposed to use the above implemented sigmoid function. This should return an array of probabilities.

[13]:
```
def sigProg(X, w, b):
    # Compute linear combination
    t = X @ w + b
    # Apply sigmoid function (already defined above)
    p = sigmoid(t)
    return p
```

Q7. Compute loss gradient.

Recall that the loss function for logistic regression is given by,

$L(\omega, b) = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2 + \lambda reg(\omega)$ where is the sigmoid probability $p_i = P(y_i = 1|x_i)$ for the data point (x_i, y_i) and $reg(\omega)$ is the regularization term - it could be either ridge $reg(\omega) = ||\omega||_2^2$ or lasso $reg(\omega) = ||\omega||_1$ or no regularization at all. $\lambda \geq 0$ is the regularization constant. N is the number of datapoints.

Then, the gradient of the loss function with respect to ω and the derivative with respect to b is given by,

$$\nabla_\omega L = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)p_i' x_i \ + \ \lambda reg'(\omega)$$

$$\frac{\partial L}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)p_i'$$

Here,

$$reg'(\omega) = \begin{cases} \omega & \text{if ridge} \\ sign(\omega) & \text{if lasso} \end{cases}$$

Implement a function named gradient to compute the gradient and derivative of the loss function, given data matrix X, output vector y, weight vector ω and bias b. The function should also be able to take in other parameters such as reg (= "none", "ridge" or "lasso") and lambda. It should return the gradient and derivative. All these computations can be done without a single for loop,

only by using numpy builtins for matrix computations, which is the most desired way to achieve efficiency

[14]:
```
def gradient(X, y, w, b, reg = "none", Lambda = 0.1):
    N = X.shape[0]
    # Compute probabilities and their derivatives
    p = sigProg(X, w, b)
    p_prime = derivSigmoid(p)
    # Compute error term
    error = (p - y) * p_prime
    # Gradient w.r.t weights
    grad_w = (X.T @ error) / N
    # Add regularization gradient if needed
    if reg == "ridge":
        grad_w += Lambda * w
    elif reg == "lasso":
        grad_w += Lambda * np.sign(w)
    # Gradient w.r.t bias
    deriv_b = np.sum(error) / N
    return grad_w, deriv_b
```

Q8. Gradient descent algorithm.

Recall the following update rule for gradient descent,

$\omega \leftarrow \omega - \eta \nabla_{\omega} L$

$b \leftarrow b - \eta \frac{\partial L}{\partial b}$

where $\eta > 0$ is the learning rate.

Implement the gradient descent algorithm in a function grad_descent given the gradient vector $\nabla_{\omega} L$ (grad_w), derivative $\frac{\partial L}{\partial b}$ (deriv_b), weights ω and bias b. Also should be able to accept the learning rate η (eta). The function should return the updated ω and b.

[15]:
```
def grad_descent(grad_w, deriv_b, w, b, eta = 0.01):
    # Update weights and bias using gradient descent
    w = w - eta * grad_w
    b = b - eta * deriv_b
    return w, b
```

Q9. Train model

Implement the function train. It should initialize ω and b to some random values or zeros. Then interatively update the ω and b using gradient descent, untill the number of interations reach the maximum number of interations specified as max_iter. The function will take in training data (X, y), regularization details (type of reg, and λ), and learning rate η (eta). Finally, it should return the trained ω and b.

[20]:
```
def train(X, y, reg = 'none', Lambda = 0.1, eta = 0.01, max_iter = 1000):
    # Initialize weights and bias
```

```
    w = np.zeros(X.shape[1])
    b = 0.0
    # Iteratively update weights and bias
    for _ in range(max_iter):
        grad_w, deriv_b = gradient(X, y, w, b, reg, Lambda)
        w, b = grad_descent(grad_w, deriv_b, w, b, eta)
    return w, b
```

Q10. Predict using the model

Implement a function predict to output predictions for given input data X, trained weights ω and b. Make sure that the output should only consist of 1's and 0's. The function should return the predictions $\hat{y}$ (yhat).

[21]:
```
def predict(X, w, b):
    # Compute probabilities using sigProg
    p = sigProg(X, w, b)
    # Convert probabilities to binary predictions (0 or 1)
    yhat = (p >= 0.5).astype(int)
    return yhat
```

Q11. Use the above logistic regression algorithm on the diabetes dataset. Train a model (ω, b). Then evaluate it. Use classification error to evaluate it.

Start by separating the diabetesData into input (features) and output (Outcome), and store them in numpy matrices X_train and y_train respectively, and then normalizing the input features.

Some tips to improve accuracy.

1. Use cross validation to find the best hyperparameters η, λ, and regularization type.
2. Plot the loss function versus iterations while training (you can do this by additionally collecting the loss values within the train function), having a smooth decreasing graph will indicate proper training.
3. As above plot the ℓ_2-norm of the ω vector versus iterations, we expect smooth decreasing curve for this as well.

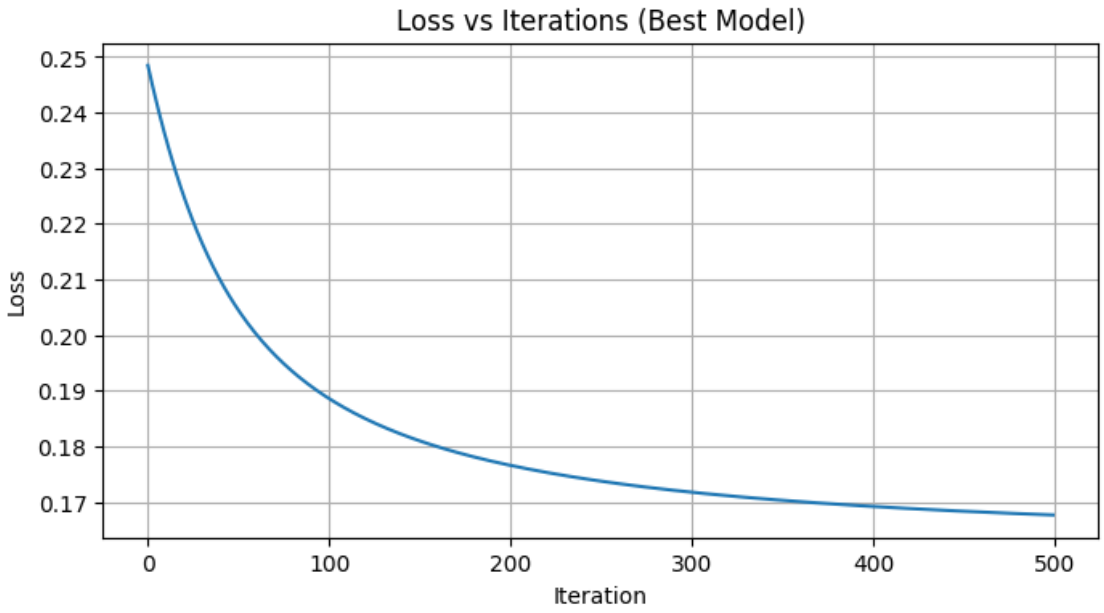
More tips and additional clarifications in class.

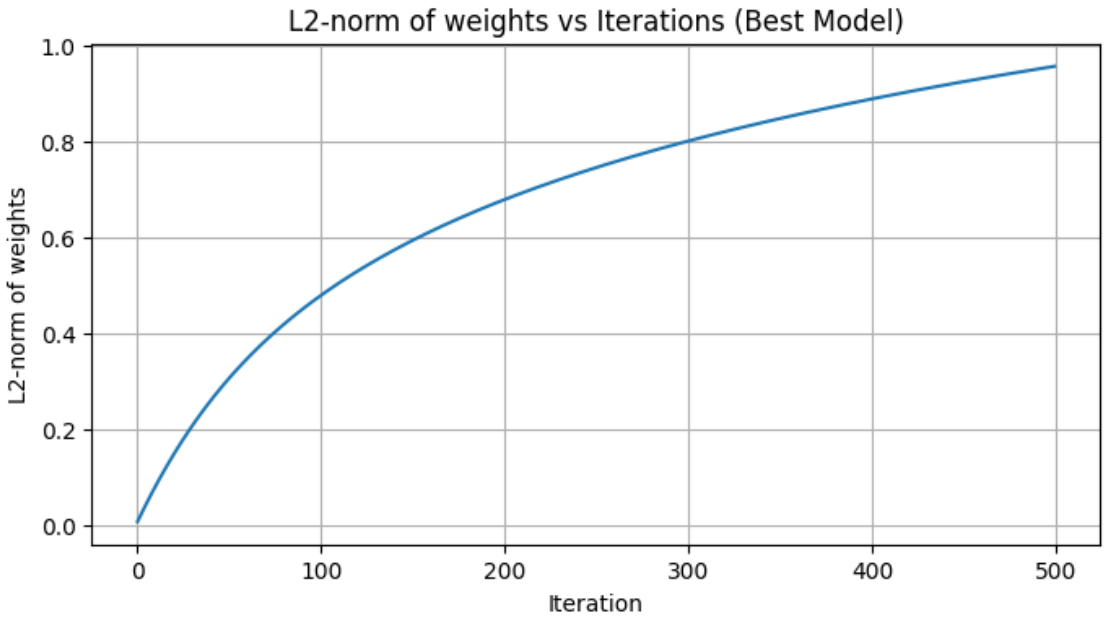
[23]:
```
from itertools import product
import matplotlib.pyplot as plt

# Prepare training data before cross-validation
X_train = diabetesData.drop('Outcome', axis=1).values
y_train = diabetesData['Outcome'].values
X_train_norm, mean_train, std_train = normalizeData(X_train)

# Hyperparameter grid for cross-validation
etas = [0.001, 0.005, 0.01, 0.05, 0.1]
lambdas = [0.001, 0.01, 0.05, 0.1, 0.5, 1.0]
regs = ['none', 'ridge', 'lasso']
```

```
best_error = float('inf')
best_params = None
best_w = None
best_b = None
best_loss_hist = None
best_norm_hist = None

def train_with_history(X, y, reg='none', Lambda=0.1, eta=0.01, max_iter=1000):
    w = np.zeros(X.shape[1])
    b = 0.0
    loss_hist = []
    norm_hist = []
    for _ in range(max_iter):
        grad_w, deriv_b = gradient(X, y, w, b, reg, Lambda)
        w, b = grad_descent(grad_w, deriv_b, w, b, eta)
        # Compute loss and norm for plotting
        p = sigProg(X, w, b)
        loss = np.mean((p - y) ** 2) + Lambda * (np.sum(w ** 2) if reg ==␣
 ↪'ridge' else np.sum(np.abs(w)) if reg == 'lasso' else 0)
        norm = np.linalg.norm(w)
        loss_hist.append(loss)
        norm_hist.append(norm)
    return w, b, loss_hist, norm_hist

# Cross-validation loop
for eta, Lambda, reg in product(etas, lambdas, regs):
    w_cv, b_cv, loss_hist, norm_hist = train_with_history(X_train_norm,␣
 ↪y_train, reg=reg, Lambda=Lambda, eta=eta, max_iter=500)
    y_train_pred_cv = predict(X_train_norm, w_cv, b_cv)
    error_cv = np.mean(y_train_pred_cv != y_train)
    if error_cv < best_error:
        best_error = error_cv
        best_params = (eta, Lambda, reg)
        best_w = w_cv
        best_b = b_cv
        best_loss_hist = loss_hist
        best_norm_hist = norm_hist

print(f"Best params: eta={best_params[0]}, Lambda={best_params[1]},␣
 ↪reg={best_params[2]}")
print(f"Best classification error: {best_error}")
print(f"Best classification accuracy: {1 - best_error}")

# Plot loss vs iterations
plt.figure(figsize=(8,4))
plt.plot(best_loss_hist)
```

```
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Loss vs Iterations (Best Model)')
plt.grid(True)
plt.show()

# Plot l2-norm of weights vs iterations
plt.figure(figsize=(8,4))
plt.plot(best_norm_hist)
plt.xlabel('Iteration')
plt.ylabel('L2-norm of weights')
plt.title('L2-norm of weights vs Iterations (Best Model)')
plt.grid(True)
plt.show()
```

```
Best params: eta=0.1, Lambda=0.001, reg=lasso
Best classification error: 0.24310776942355888
Best classification accuracy: 0.7568922305764412
```





Q12. Testing on previously unseen test data - final part.

1. Load the DiabetesTest.csv file.
2. Use your best trained model to predict the 'Outcome' for this test data, assign your predictions to y_test_pred, this should be an array of 0's and 1's.
3. This will be graded on the level of accuracy of your predictions.
4. Remember to properly normalize your data before using them in the model. For normalization use the mean and standard deviation of the training data not the test data.

[19]:
```
# Load the test data
diabetesTest = pd.read_csv('DiabetesTest.csv')
# Display the first few rows of the dataset
print(diabetesTest.head())

# Separate features from test data (exclude Outcome if present)
if 'Outcome' in diabetesTest.columns:
    X_test = diabetesTest.drop('Outcome', axis=1).values
else:
    X_test = diabetesTest.values

# Normalize test features using training mean and std
X_test_norm, _, _ = normalizeData(X_test, mean_train, std_train)

# Predict outcomes for test data using best model from cross-validation
y_test_pred = predict(X_test_norm, best_w, best_b)

# Print predictions for review
```

```
print('Test predictions:', y_test_pred)
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            1       89             66             23       94  28.1
1            3       78             50             32       88  31.0
2            7      147             76              0        0  39.4
3            1       97             66             15      140  23.2
4            4      111             72             47      207  37.1

   DiabetesPedigreeFunction  Age
0                     0.167   21
1                     0.248   26
2                     0.257   43
3                     0.487   22
4                     1.390   56
Test predictions: [0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 1 0
 0 0 0 0 0 0 1 1 0 1 1 1 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 1 0 0
 0 0 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1]
```